

Major Web Application Project - Student Brief (v0.9)

Weight: 90% of course grade | Individual | PHP, Laravel, SQLite, HTML/CSS | Milestones: Weeks 8-12

This assessment rewards a coherent, usable application that you understand and can defend - not a checklist of disconnected features. Point values and time budgets are a starting calibration.

1. Overview & AI policy

You will design, build, and deploy a database-driven web application of your choosing (e.g. an e-commerce store, task manager, or student portal).

State your app's purpose in one sentence.

Before you build, write a single sentence: "This app helps [who] to [do what]." Every feature you build should serve that purpose.

AI as co-pilot.

You are permitted and encouraged to use AI to accelerate development. But you are the lead engineer and you hold full liability for your code. Never submit code you cannot explain. Your grade is set largely by what you can defend, not by what you submit.

2. How you'll be graded

Final (out of 90) = 90 × Understanding × Complexity

Complexity - additive (what you built)

Core requirements (1-5).

The five core requirements (1-5) below contribute UP TO 0.60. Each contributes 0.12 for a complete implementation, 0.06 for a materially partial one, or 0 if absent. Feature points are added to the core points you actually earn, total capped at 1.00. (So completing four of the five core items gives 0.48, not 0.60.) If authentication, server-side validation, or ownership authorization is entirely absent, Complexity is additionally capped at 0.65 until it is fixed.

#	Core requirement
1	Authentication (register / login / logout), with the logged-in user's name or username displayed on every page.
2	Relational database: 3+ related tables via Eloquent, related by foreign keys, with the relationships defined AND used by the app.
3	Full CRUD (all four operations - create, read, update, delete) on a main resource.
4	Server-side validation appropriate to each field: every user-editable field is validated on the server according to its data type and constraints - required fields reject empty input; numbers, dates and emails reject the wrong type/format; bounded fields enforce their range or length (e.g. a 1-5 rating rejects 6); references reject non-existent IDs; file uploads check type and size. A field with no real constraints may simply be 'required'.
5	Ownership authorization enforced on the server for at least update and delete (and viewing when records are private): it must block a user who crafts a request or edits the URL to reach another user's record - hiding the Edit/Delete button is not enough.

Feature menu (6-15) - added to the core points you earn, total capped at 1.00. Requirements are numbered in one continuous sequence: core 1-5, features 6-15, so you and the marker can refer to any of them by number (e.g. "feature 11").

A feature earns its points only when it is complete and actually used in the app - the full round trip, not a fragment.

#	Feature	Adds
6	Image / file upload — users upload an image or file through a form; it is stored and then displayed (image) or downloadable via a working link (file). The full upload-and-use round trip, not just a file on the server.	+0.05
7	Pagination combined with search / filtering — a list that is both searchable/filterable and paginated, working together: search/filter narrows the results, the narrowed results paginate, and the search/filter is preserved across pages.	+0.05
8	Coherent & usable app — serves its stated purpose; features integrated; navigable; applies the Week-1 front-end skills (judged in the user-journey demo).	+0.05
9	4th related table — related by a foreign key and actively used by the app (read and written as part of a working feature), not an orphaned table.	+0.03
10	5th related table — related by a foreign key and actively used by the app.	+0.03
11	Many-to-many via a pivot table — users can create the links (e.g. add tags to a post, items to a cart) and see them displayed; not just a pivot declared in the model.	+0.07
12	Roles + middleware — two or more roles with genuinely different permissions, enforced by middleware, protecting a real role-restricted area or action; the restriction actually blocks access when tested.	+0.07
13	Asynchronous UI (AJAX/Livewire) — changes data on the server and updates the page without a full reload; the database actually changes and the UI updates in place, not a cosmetic change.	+0.07
14	3+ passing automated feature tests — exercise real behaviour (create a record, validation rejecting bad input, a protected route requiring login) and pass when run; not trivial or placeholder tests.	+0.07
15	Third-party API integration — your app calls a genuine third-party API and uses the result as part of a real feature (displayed, stored, or acted upon), not an unused call.	+0.08

The integration rule.

A feature earns its points only when it is part of your app's core workflow, not a standalone demo.

Understanding - judged in the viva

Each band maps to a single multiplier (no ranges), so marking is consistent:

Band	Multiplier	Description
Fluent	1.00	Explains own and AI-assisted code precisely, AND makes the live change quickly and unaided.
Solid	0.85	Explains the data flow correctly, AND makes the live change with only minor difficulty.
Shaky	0.70	Explains only with heavy prompting; OR explains reasonably but completes the live change only with substantial help, or not at all.
Poor	0.45	Struggles to explain their own code and makes little progress on the change even with major help.
Fail	0.20	Cannot navigate or explain the codebase and cannot make the change; or leaves severe vulnerabilities unaddressed.

The live change is the decisive signal - the hardest part to fake. A student who explains fluently but cannot make the live change is placed no higher than Shaky (0.70).

Writing thorough comments is good practice - but reading your comments back is not the same as understanding. You will be asked to explain in your own words (the why, not just the what) and to make a live change; comments do neither for you.

3. Coherence & usability

- Coherent: your features work together to serve your one-sentence purpose. Features that work only in isolation do not count toward Complexity.
- Usable: navigable, labelled forms, applying the HTML/CSS/layout skills from Week 1. Basic usability is expected; visual polish beyond that is not graded, and its absence beyond “usable” costs nothing - but a barely navigable app falls short of the “real web application” bar.

Both are checked in the viva by a real user-journey: you complete one genuine end-to-end task a user of your app would do.

4. What to submit - the Codebase Map (1-page PDF, with your code)

The Codebase Map is required, and it is how you claim your grade. If you do not submit one, your Complexity is capped at core (0.60) - the marker will not hunt for features, and the viva will NOT be extended to let you list them verbally. It must contain:

- Your one-sentence app purpose.
- Target grade and your resulting Complexity figure (core points earned + the features you claim).
- Feature table, ordered highest-value first, using the requirement numbers above, each with its location (e.g. “Feature 11 (pivot - cart) - CartController@store, lines 20-48”).
- Proudest code: the one feature you most want to be examined on.
- AI log: where AI helped you most, and one concrete instance where AI produced a bug or bad architecture that you found and fixed.
- For each advanced feature (API, async UI, tests, pivot, upload...), propose 1-2 small changes a marker could ask you to make, and where in the code they would go. Be ready to perform one live, without AI.

5. Submission & the Week-12 demonstration

Authoritative submission (due Friday, Week 11)

Submit to the LMS a zip of your project and your Codebase Map (PDF). The project zip you submit by the deadline is the authoritative assessed version.

Demonstrating from a prepared copy

You may demonstrate using a prepared local copy so your application is ready to run when called, but your local demonstration copy must materially match the LMS-submitted version.

The marker may compare relevant source files with your LMS submission where there is an apparent discrepancy, or as part of routine random verification.

Environment-specific and generated differences - such as .env settings, dependencies (vendor), caches, logs, database contents, and compiled files - are permitted. Changes to assessed source code or functionality after the deadline are not permitted.

Perform any live change (the viva mutation) in a separate temporary copy of the project; the mutation is part of the viva and is not treated as a post-deadline change to your submission. If you want to review your code or rehearse beforehand, also use a separate copy.

Before your turn (pre-flight)

- Your app already running from your prepared copy.
- Your Codebase Map and your code editor open.

The viva (~12-15 minutes)

1. User-journey demo (up to 3 minutes) - you complete one real end-to-end task, chosen in advance, that a user of your app would do (this shows your core works, and that the app is coherent and usable). Keep it focused; the tutor may move on once they have seen enough.
2. Explain-and-audit - the tutor picks one feature at your grade boundary and one of your features at random; for each you demonstrate it, explain the code in your own words, and answer one data-flow or “what if a user entered malicious data here?” question.
3. Live micro-mutation (about 4 minutes) - a small change to your own code, on the spot, without generative AI.

The live change is deliberately small - the kind you can make in a few minutes with your editor already open (which is why the pre-flight has you ready to go). You have about 4 minutes, and it will resemble the candidate mutations you listed in your Codebase Map. If you don't complete the first change, you will be offered a second, simpler one on a different part of your code. Your Understanding band reflects your overall ability to modify your own code across the viva - not a single attempt. The time allowed is the same for every student.

6. How marking works & academic integrity

Marking is trust-but-verify. You declare your features; the tutor samples the grade-deciding ones plus one at random, rather than checking everything. A feature counts toward Complexity when it is genuinely present and used. If you claim a feature that isn't there, it earns nothing - and if there is evidence you deliberately misrepresented your work, that is a separate academic-integrity matter. A feature that is present but you cannot explain still counts as built, but a weak defence lowers your Understanding multiplier, which is where most of the mark is won or lost. Declare honestly and build what you understand.

7. Grading examples

- Blind-AI student. Builds an HD feature set but cannot explain the API and fails the live change → Understanding Fail (0.20). Even with high Complexity: $90 \times 0.20 \times 1.00 = 18$ (fail).
- Feature-collector. Ticks many features, but they are disconnected demos; non-integrated ones aren't credited and the user-journey fails, so Complexity falls toward core.
- Honest core, fluent. Complexity 0.60, Understanding Fluent (1.00) → 54 (60%, pass).
- Honest core, solid. Complexity 0.60, Understanding Solid (0.85) → 45.9 (51%, pass) - a complete, working, well-explained core app passes.
- Mixed-features. Core + feature 6 (upload) + feature 11 (pivot) = 0.72, Understanding Solid (0.85) → 55 (61%).
- Empowered developer. Full feature set (Complexity 1.00), Understanding Solid (0.85) → 76.5 (85%, HD).

8. Milestones

M1 Blueprint (one-sentence purpose, ERD, wireframes, routes map - checked live in Weeks 7-9). M2 Codebase + Codebase Map (due Friday Week 11). M3 Viva (Week 12).